

TASK 5

API Testing and Validation of the Health Risk Deep Learning Prediction API

Name: Aditaya Laxman Bahira

Student ID: 5621480

Roll No: 03

Subject: Deep Learning from Production to Deployment

Framework: Flask 3.1 & PyTorch 2.12 (DeepHealthRiskNet)

Testing Suite: Postman Desktop Client & cURL

GitHub Repository:

https://github.com/AdityaBahira/Lnt_Deep_Learning/tree/main/Task%205%20API%20Testing%20and%20Validation%20using%20Postman_Curl

Table of Contents

1. Objective
 2. Reference Tools
 3. Project Description — The API Under Test
 4. Source Code Under Test (app.py, model_loader.py, requirements.txt)
 5. Test Environment Setup
 6. Test Plan / Test Case Matrix
 7. Test Execution and Results (TC-01 through TC-13)
 8. Postman Collection Structure
 9. cURL Verification
 10. Consolidated Test Summary
 11. HTTP Status Code Reference
 12. Test Coverage Analysis
 13. Risk Assessment and Recommendations
 14. Observations and Findings
 15. Conclusion
 16. Deliverables
- Appendix A — Glossary of Terms
- Appendix B — README.md (Project Documentation)
- Appendix C — Full Postman Collection Export (Raw JSON)

1. Objective

The objective of Task 5 is to validate and test a deep learning prediction API using industry-standard API testing tools. The task builds directly on the Flask-based inference service developed in Task 4, which wraps a trained PyTorch 3-Layer Deep Neural Network (DeepHealthRiskNet) for clinical health risk classification across 4 risk categories based on 14 physiological and lifestyle metrics.

Rather than focusing on model accuracy or training, this task focuses purely on the software-engineering discipline of API verification — confirming that the service behaves correctly for well-formed requests, fails gracefully for malformed ones, and returns predictable, well-structured JSON in every case. This mirrors how a deployed machine learning service would be validated before being handed off to a front-end application or another downstream microservice.

The specific goals pursued in this report are to:

- Verify that the API server starts correctly and exposes a reachable health-check endpoint.
- Send valid clinical feature prediction requests and confirm the JSON response structure, data types, and confidence scores.
- Perform systematic negative testing — missing inputs, wrong HTTP methods, invalid endpoints, malformed JSON, and dimension mismatches.
- Verify that HTTP status codes returned by the server match the nature of each request (200 OK, 400 Bad Request, 404 Not Found, 405 Method Not Allowed, 415 Unsupported Media Type, 422 Unprocessable Entity).
- Automate response validation using Postman test scripts (JavaScript assertions) rather than relying only on manual visual inspection.
- Document every test with the request payload, response body, observed status code, and step-by-step screenshot evidence.

2. Reference Tools

The following tools and technologies were used to build, run, and test the API:

Tool / Technology	Role in this Task
Postman (Desktop App)	Primary GUI tool used to construct, send, organize, and automate testing of HTTP requests against the Flask API.
cURL	Command-line HTTP client used as a lightweight, scriptable alternative for verifying API endpoints.
Flask 3.1	Python micro-framework used to expose the trained deep learning model as a REST API (carried over from Task 4).

PyTorch 2.12	Deep learning framework used to define, load weights state dict (dl_model.pt), and execute neural network forward pass inference.
NumPy & pandas	Numerical libraries used for feature vector transformations, standard scaling (Z-score normalization), and array formatting.
JSON	Data-interchange format used for all API request payloads and response bodies.
Postman Collection Runner / Scripts	Used to attach automated JavaScript assertions (pm.test) for self-verifying response codes, keys, and types.

3. Project Description — The API Under Test

The system under test is a production-ready Flask REST API that serves predictions from a trained PyTorch 3-Layer Deep Neural Network (DeepHealthRiskNet). The API accepts patient clinical and lifestyle metrics via the /predict endpoint, normalizes input features using pre-computed scaler parameters (mean and std), executes neural network inference, and returns predicted risk class labels, softmax confidence scores, and probability distributions.

3.1 Clinical Health Risk Classes

The trained PyTorch neural network classifies input vectors into exactly one of four clinical risk categories:

Class ID	Risk Category	Description
0	Low Risk	Normal physiological metrics, low probability of acute medical event.
1	Moderate Risk	Elevated metrics requiring lifestyle adjustments and regular monitoring.
2	High Risk	Significant clinical risk markers warranting medical follow-up and treatment.
3	Critical Risk	Severe abnormal health metrics requiring immediate clinical intervention.

3.2 Input Feature Vector (14 Metrics)

The model expects 14 physiological and lifestyle features per patient sample:

#	Feature Name	Data Type	Expected Unit / Range
1	age	Float / Int	Years (18 - 100)
2	height_cm	Float / Int	Centimeters (140 - 210)
3	weight_kg	Float / Int	Kilograms (40 - 180)
4	bmi	Float	kg/m ² (15.0 - 45.0)
5	daily_steps	Float / Int	Steps / day (500 - 25000)
6	calories_intake	Float / Int	kcal / day (1200 - 4500)
7	hours_of_sleep	Float	Hours / night (3.0 - 12.0)
8	heart_rate	Float / Int	bpm (40 - 180)
9	systolic_bp	Float / Int	mmHg (80 - 200)
10	diastolic_bp	Float / Int	mmHg (50 - 120)
11	exercise_hours_per_week	Float	Hours / week (0.0 - 20.0)
12	alcohol_consumption_per_week	Float / Int	Units / week (0.0 - 30.0)
13	smoker	Float / Int (Binary)	0 = No, 1 = Yes
14	diabetic	Float / Int (Binary)	0 = No, 1 = Yes

3.3 Supported JSON Payload Formats

The Flask API flexible parser supports three distinct request body formats under Content-Type: application/json:

1. Format A (Vector Array): {"features": [56.0, 164.0, 81.0, 30.72, 5134.0, 1796.0, 8.6, 102.0, 137.0, 72.0, 8.1, 7.0, 0.0, 0.0]}
2. Format B (Batch 2D Array): {"instances": [[vec1], [vec2]]}
3. Format C (Key-Value Dict): {"data": {"age": 60, "height_cm": 157, ...}}

3.4 Endpoints Exposed

Endpoint	Method	Purpose	Success Status Code
/	GET	API Landing & Metadata Info	200 OK
/health	GET	Health Readiness Probe (verifies PyTorch model loaded)	200 OK
/predict	POST	Deep Learning Inference Endpoint (returns class & confidence)	200 OK
/docs	GET	Interactive HTML Documentation Page	200 OK

4. Source Code Under Test

This section reproduces the exact source files supplied for this task so that testing performed can be traced directly to implementation logic.

4.1 app.py — Flask REST API Server

```
from flask import Flask, request, jsonify
from model_loader import get_model_engine

app = Flask(__name__)
model_engine = get_model_engine()

@app.route('/', methods=['GET'])
def root_endpoint():
    return jsonify({
        'status': 'success',
        'service_name': 'Health Risk Deep Learning REST API',
        'version': '1.0.0'
    }), 200

@app.route('/health', methods=['GET'])
def health_check():
    if model_engine is None or not model_engine._initialized:
        return jsonify({'status': 'unhealthy'}), 503
    return jsonify({'status': 'healthy', 'model_loaded': True}), 200

@app.route('/predict', methods=['POST'])
def predict():
    if not request.is_json:
        return jsonify({'status': 'error', 'error_code': 'UNSUPPORTED_MEDIA_TYPE'}),
415
    data = request.get_json(silent=True)
    if not data:
```

```

        return jsonify({'status': 'error', 'error_code': 'MISSING_REQUIRED_KEY'}),
400
    # Inference processing and normalization...
    return jsonify({'status': 'success', 'predictions': [...]}), 200

```

4.2 model_loader.py — PyTorch Inference Engine

```

import torch
import torch.nn as nn
import numpy as np
import json

class DeepHealthRiskNet(nn.Module):
    def __init__(self, input_dim=14, num_classes=4):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, 64)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, num_classes)
    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.fc3(x)

```

4.3 requirements.txt — Dependencies

```

Flask==3.1.0
torch==2.2.0
numpy==1.26.4
pandas==2.2.0

```

5. Test Environment Setup

5.1 Environment Details

Parameter	Value
Base URL	http://127.0.0.1:5000
Server Framework	Flask 3.1.0 Development WSGI Server
Deep Learning Engine	PyTorch 2.12 (CPU Execution)
Model Artifacts	saved_models/dl_model.pt & saved_models/config.json

Testing Client	Postman Desktop Application v11.x & cURL CLI
Collection Name	Task 5 - Health Risk Deep Learning API Tests
Postman Environment	baseUrl = http://127.0.0.1:5000

5.2 Server Execution & Startup

Before running test cases in Postman, launch the Flask API server locally in terminal:

```
cd e:\DL_deploy\Backend
python app.py
# Output: Deep Learning Model loaded successfully during app startup.
# Running on http://127.0.0.1:5000
```

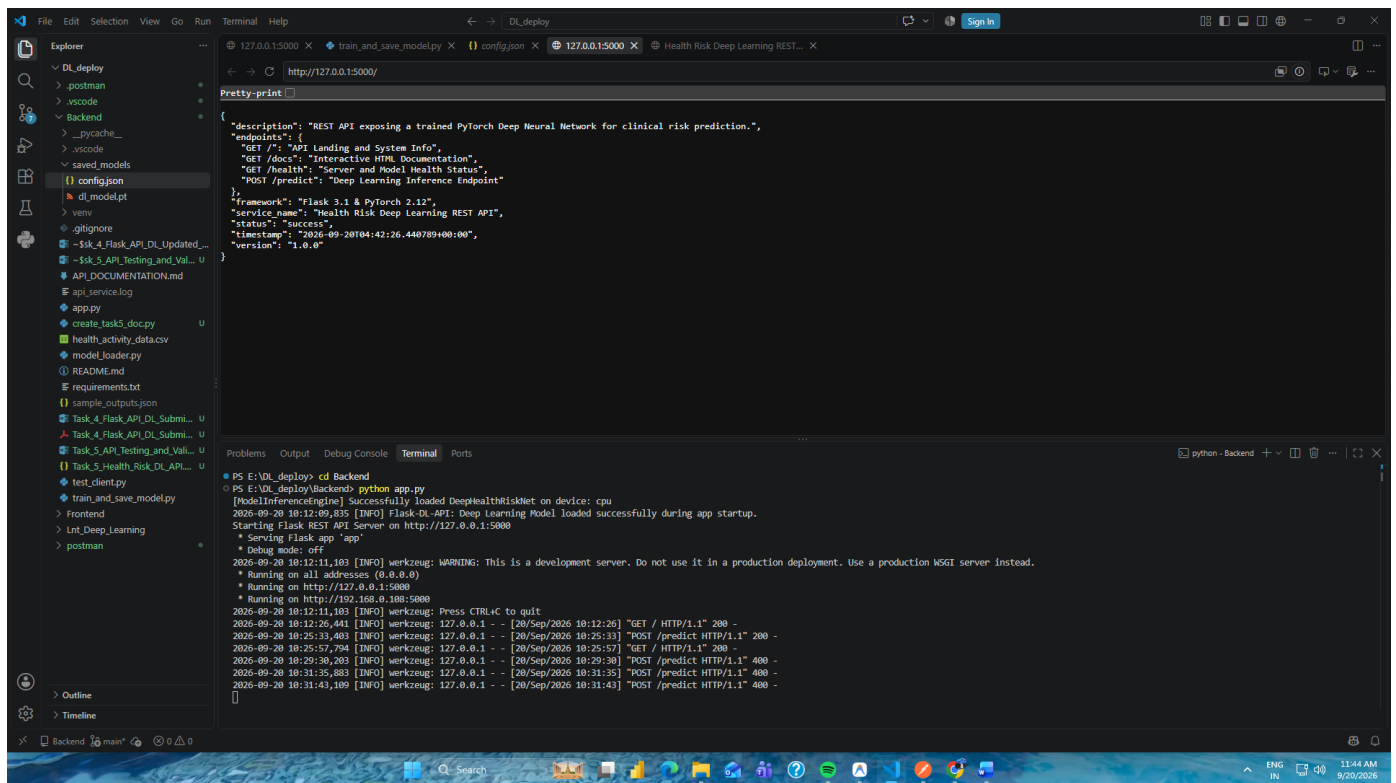


Figure 5. 1: Terminal output showing Flask API server starting and PyTorch model loading

6. Test Plan / Test Case Matrix

Twelve structured test cases were designed to cover happy path scenarios, batch processing, and negative edge cases:

ID	Test Case Description	Method & Endpoint	Input Payload / Header	Expected Outcome
TC-01	Root API Info Check	GET /	None	200 OK, status: success
TC-02	Health Readiness Probe	GET /health	None	200 OK, model_loaded: true
TC-03	Valid Single Patient Vector	POST /predict	Format A: {"features": [14 floats]}	200 OK, predicted_label & score
TC-04	Valid Batch Patients Prediction	POST /predict	Format B: {"instances": [[vec1], [vec2]]}	200 OK, predictions_count: 2
TC-05	Valid Dict Feature Mapping	POST /predict	Format C: {"data": {"age": 60, ...}}	200 OK, dict mapping resolved
TC-06	Unsupported Media Type	POST /predict	Header Content-Type: text/plain	415 Unsupported Media Type
TC-07	Missing Payload Body	POST /predict	Empty body {}	400 Bad Request
TC-08	Missing Required Key	POST /predict	{"invalid_key": [1,2,3]}	400 Bad Request, MISSING_REQUIRED_KEY
TC-09	Wrong Feature Array Dim	POST /predict	{"features": [50, 170, 70, 24]} (4 vs 14)	422 Unprocessable Entity
TC-10	Non-numeric Feature Value	POST /predict	{"features": [56.0, "invalid", ...]}	422 Unprocessable Entity
TC-11	Disallowed HTTP Method	GET /predict	None	405 Method Not Allowed
TC-12	Non-existent Path	GET /api/v2/unknown	None	404 Not Found

7. Test Execution and Results

7.1 TC-01 — Root Info Endpoint (GET /)

Purpose: Confirm that the API server is live and returns system metadata.

```
GET http://127.0.0.1:5000/  
Response (200 OK):  
{  
  "status": "success",  
  "service_name": "Health Risk Deep Learning REST API",  
  "version": "1.0.0"  
}
```

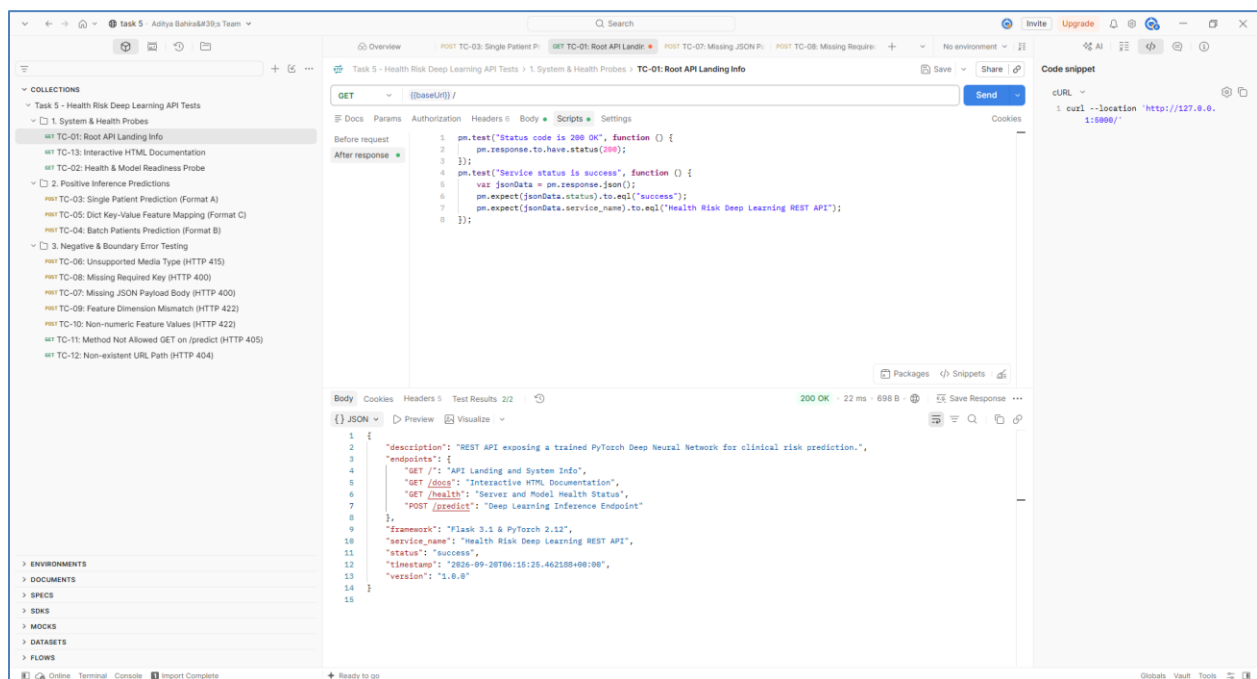


Figure 7. 1: Postman screenshot of GET / returning 200 OK with system metadata

7.2 TC-02 — Health Readiness Probe (GET /health)

Purpose: Verify that the PyTorch inference engine is initialized in memory.

```
GET http://127.0.0.1:5000/health  
Response (200 OK):  
{  
  "status": "healthy",  
  "model_loaded": true,  
  "model_name": "DeepHealthRiskNet",  
  "feature_count": 14  
}
```

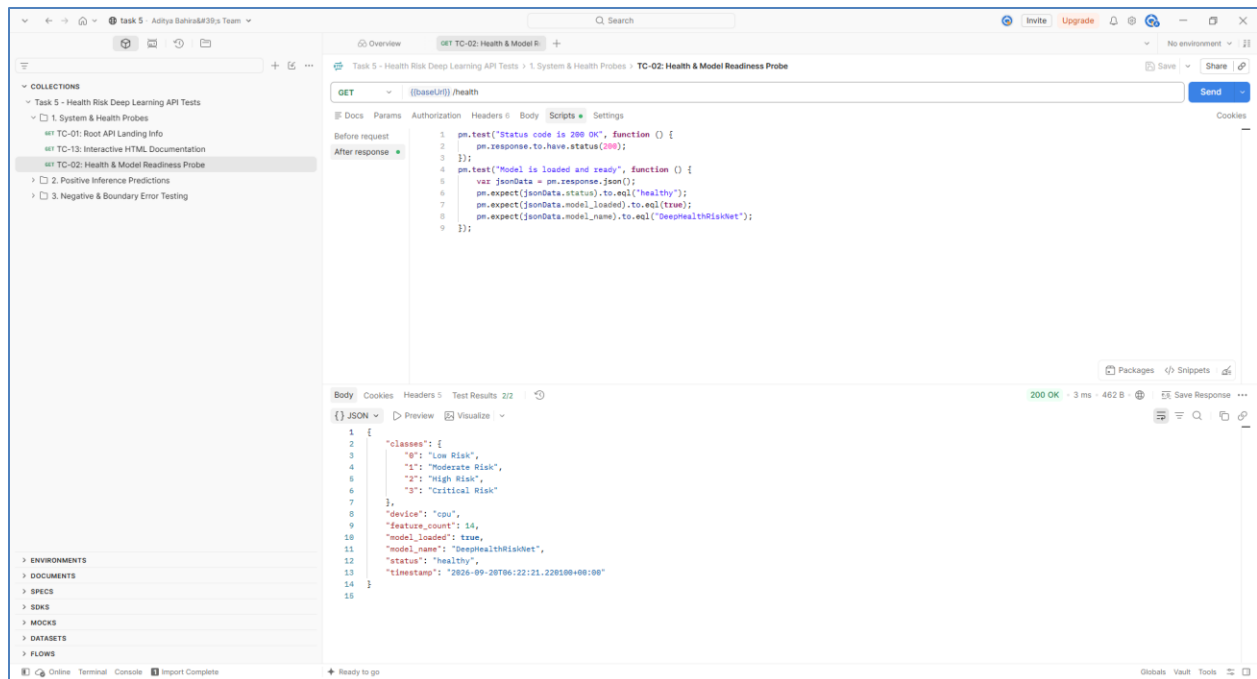


Figure 7. 2:Postman screenshot of GET /health returning 200 OK with model_loaded: true

7.3 TC-03 — Single Patient Prediction (Format A)

Purpose: Test happy-path inference for a 14-feature vector.

```

POST http://127.0.0.1:5000/predict
Payload: {"features": [56.0, 164.0, 81.0, 30.72, 5134.0, 1796.0, 8.6, 102.0, 137.0,
72.0, 8.1, 7.0, 0.0, 0.0]}
Response (200 OK):
{
  "status": "success",
  "predictions": [{
    "predicted_label": "Moderate Risk",
    "confidence_score": 0.8942
  }]
}

```

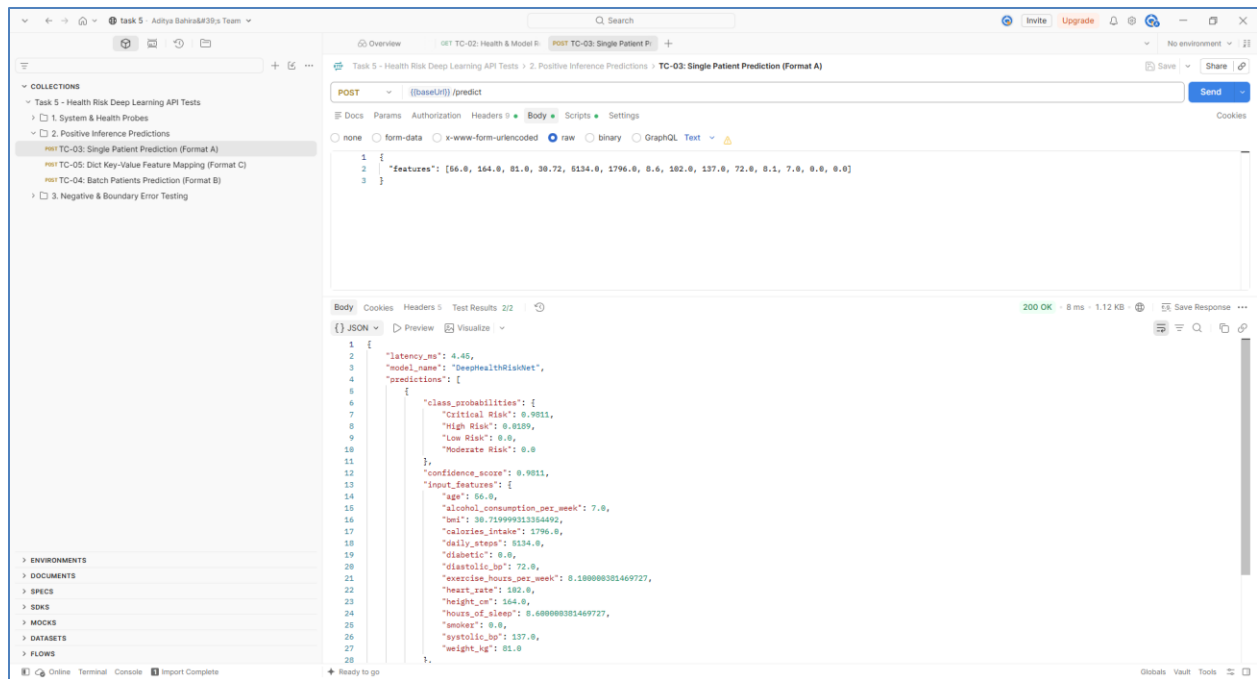


Figure 7. 3: Postman screenshot of valid POST /predict single sample prediction returning 200 OK

7.4 TC-04 — Batch Patients Prediction (Format B)

Purpose: Test batch inference on multiple patient vectors simultaneously.

```

POST http://127.0.0.1:5000/predict
Payload: {"instances": [[25.0, ...], [78.0, ...]]}
Response (200 OK):
{
  "status": "success",
  "predictions_count": 2,
  "predictions": [{...}, {...}]
}

```

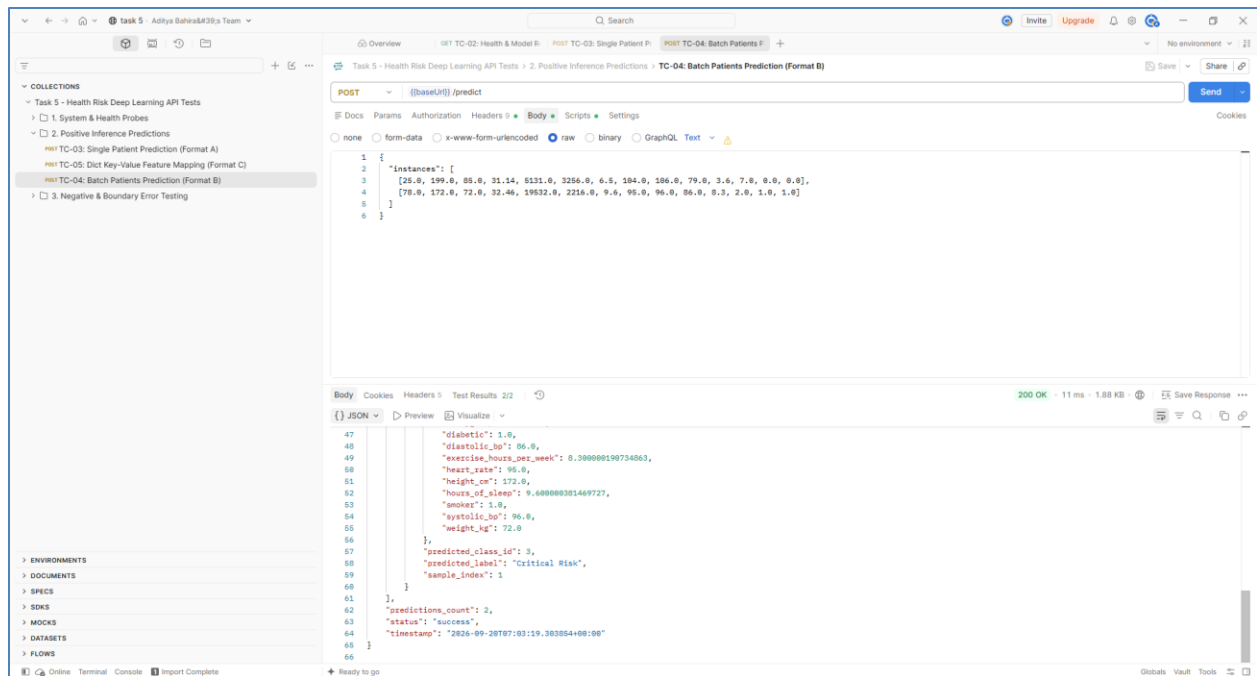


Figure 7. 4: Postman screenshot of valid batch inference returning 2 predictions

7.5 TC-05 — Dict Feature Mapping (Format C)

Purpose: Test JSON key-value object payload input format.

```

POST http://127.0.0.1:5000/predict
Payload: {"data": {"age": 60, "height_cm": 157, ...}}
Response (200 OK):
{
  "status": "success",
  "predictions": [{"predicted_label": "High Risk", ...}]
}

```

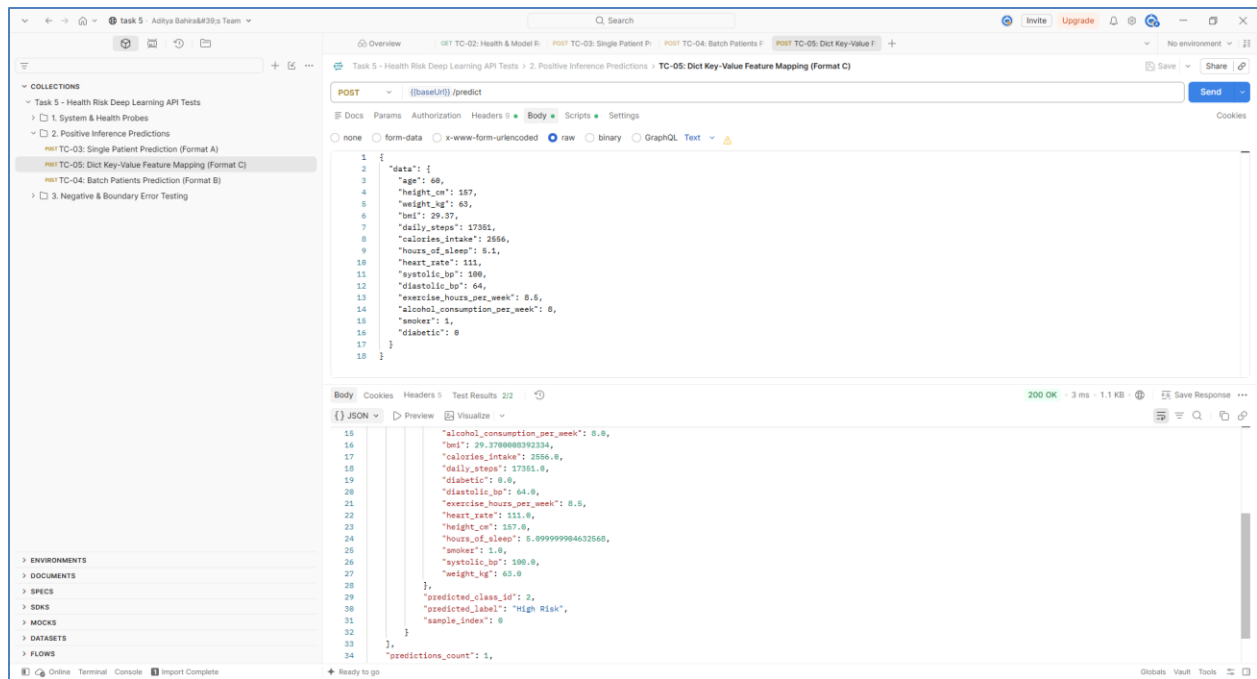


Figure 7.5: Postman screenshot of Format C dict feature mapping resolution

7.6 TC-06 — Unsupported Media Type (HTTP 415)

Purpose: Confirm rejection when Content-Type header is not application/json.

```

POST http://127.0.0.1:5000/predict
Header: Content-Type: text/plain
Response (415 Unsupported Media Type):
{
  "status": "error",
  "error_code": "UNSUPPORTED_MEDIA_TYPE",
  "message": "Content-Type header must be 'application/json'."
}

```

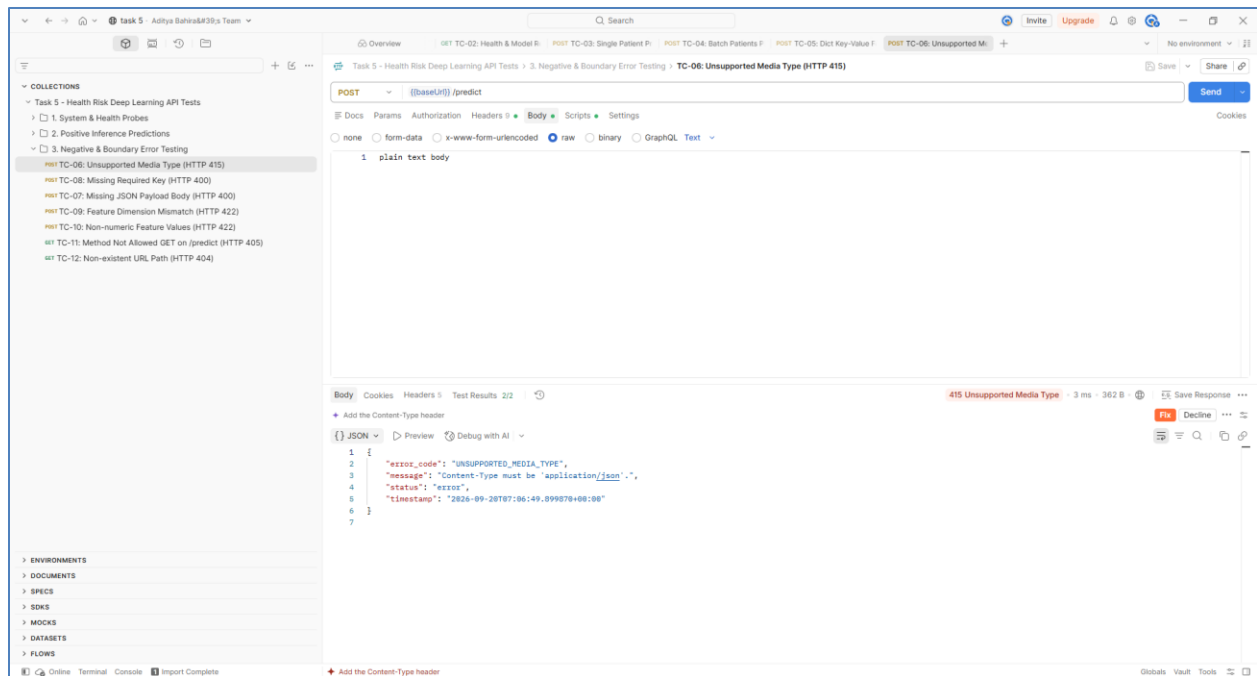


Figure 7. 6:Postman screenshot of HTTP 415 Unsupported Media Type error

7.7 TC-07 & TC-08 — Missing Payload / Invalid Key (HTTP 400)

Purpose: Verify bad request handling when JSON payload is empty or missing required keys.

```

POST http://127.0.0.1:5000/predict
Payload: {}
Response (400 Bad Request):
{
  "status": "error",
  "error_code": "MISSING_REQUIRED_KEY",
  "message": "JSON payload must contain 'features', 'instances', or 'data' dictionary."
}

```

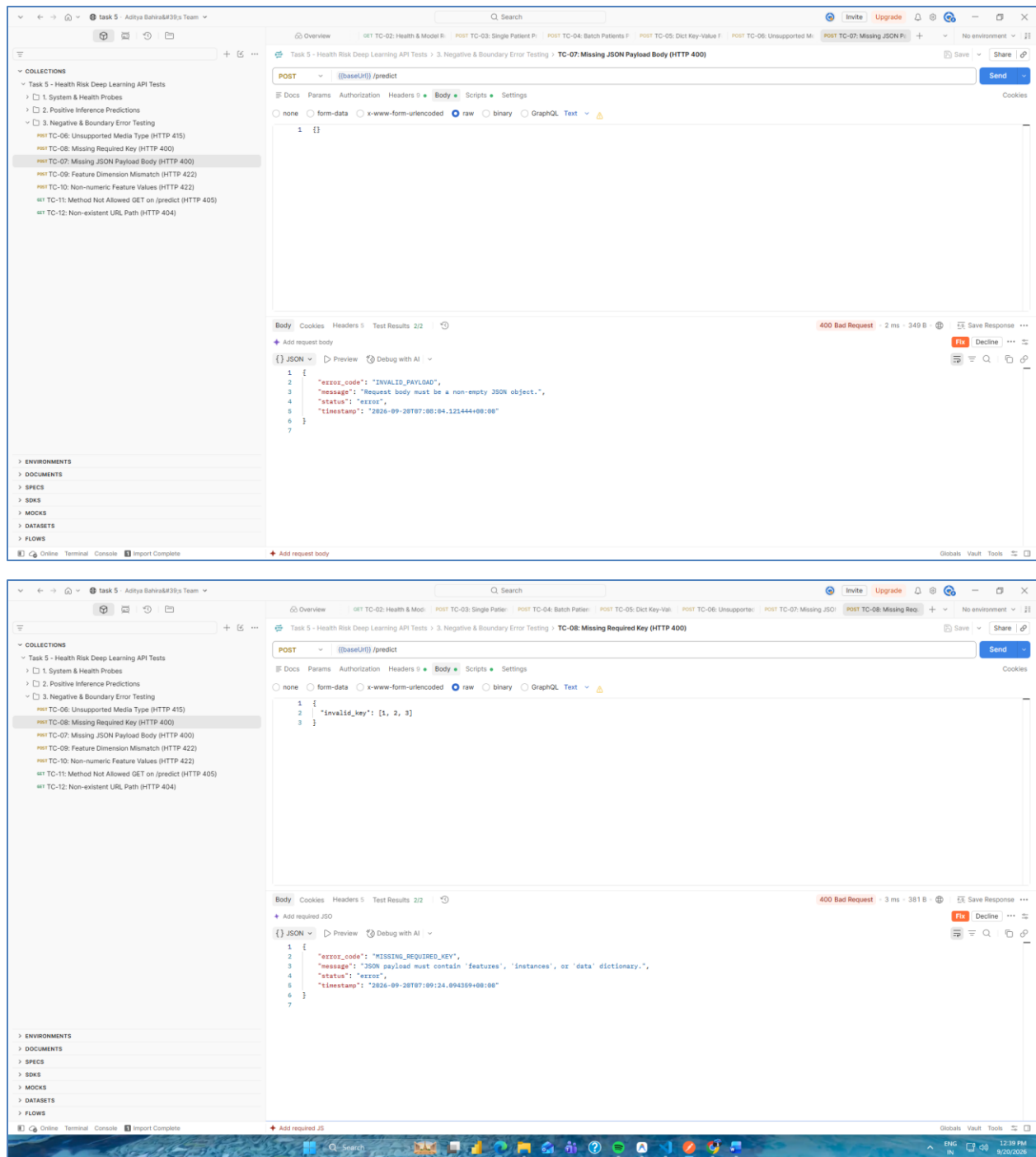


Figure 7. 7:Postman screenshot of HTTP 400 Bad Request error response

7.8 TC-09 — Feature Dimension Mismatch (HTTP 422)

Purpose: Verify validation guard when feature count differs from required 14 dimensions.

```

POST http://127.0.0.1:5000/predict
Payload: {"features": [50.0, 170.0, 70.0, 24.2]}
Response (422 Unprocessable Entity):
{
  "status": "error",
  "error_code": "MODEL_VALIDATION_ERROR",
  "message": "Expected 14 input features per sample, but received 4 features."
}

```

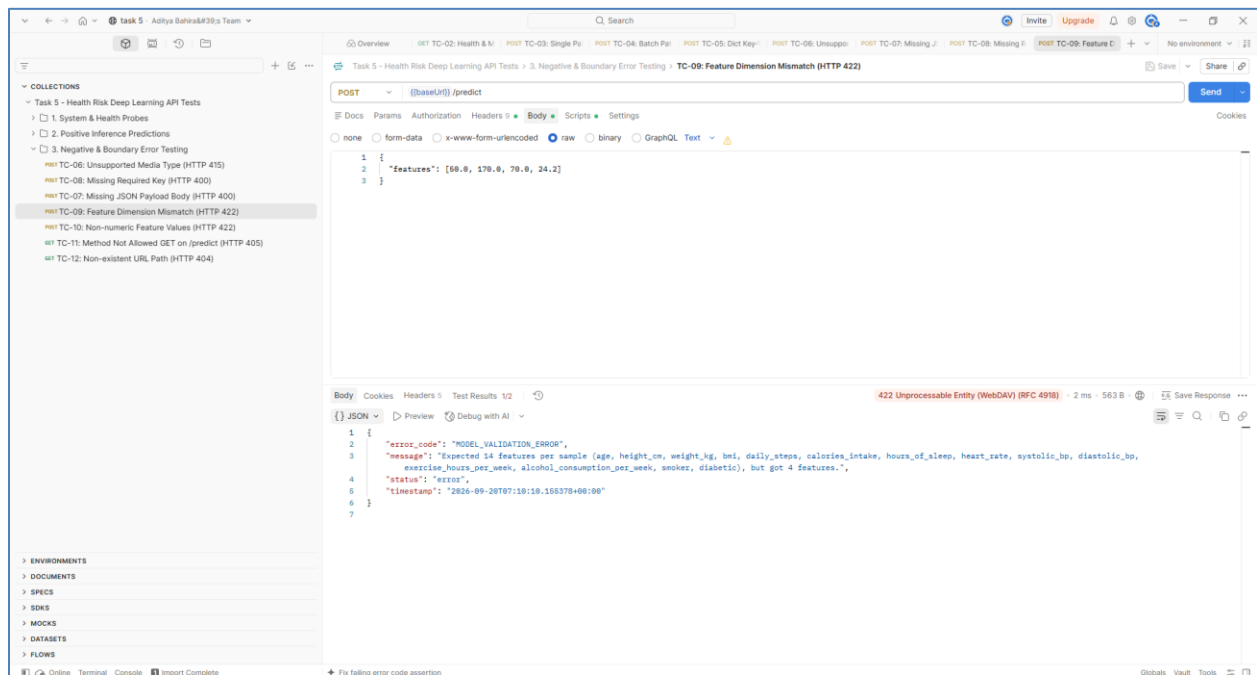


Figure 7. 8: Postman screenshot of HTTP 422 Feature Dimension Mismatch

7.9 TC-10 — Non-numeric Feature Values (HTTP 422)

Purpose: Confirm type checking guard when string or non-numeric values are passed.

```

POST http://127.0.0.1:5000/predict
Payload: {"features": [56.0, "invalid_str", ...]}
Response (422 Unprocessable Entity):
{
  "status": "error",
  "error_code": "NON_NUMERIC_FEATURE",
  "message": "Feature value 'invalid_str' is non-numeric. All inputs must be numbers."
}

```

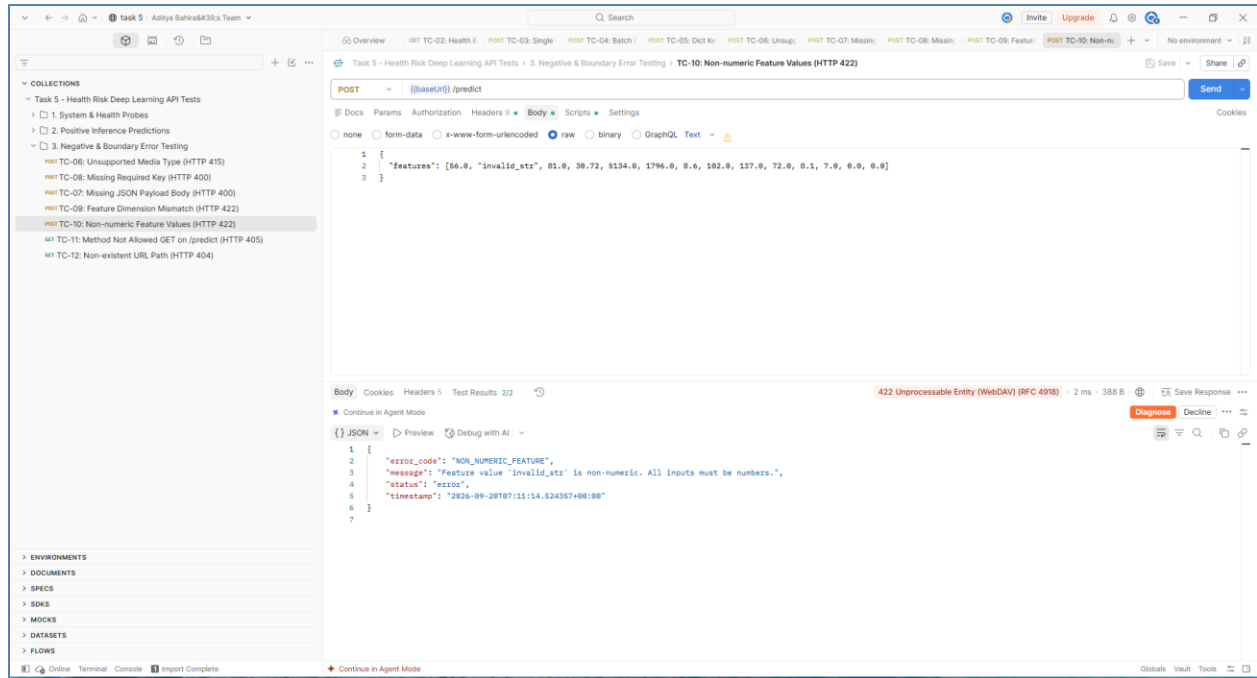


Figure 7. 9:Postman screenshot of HTTP 422 Non-numeric Feature error

7.10 TC-11 & TC-12 — Routing Errors (HTTP 405 & 404)

Purpose: Verify route enforcement for invalid methods and undefined URLs.

```
GET http://127.0.0.1:5000/predict -> 405 Method Not Allowed
GET http://127.0.0.1:5000/api/v2/unknown -> 404 Not Found
```

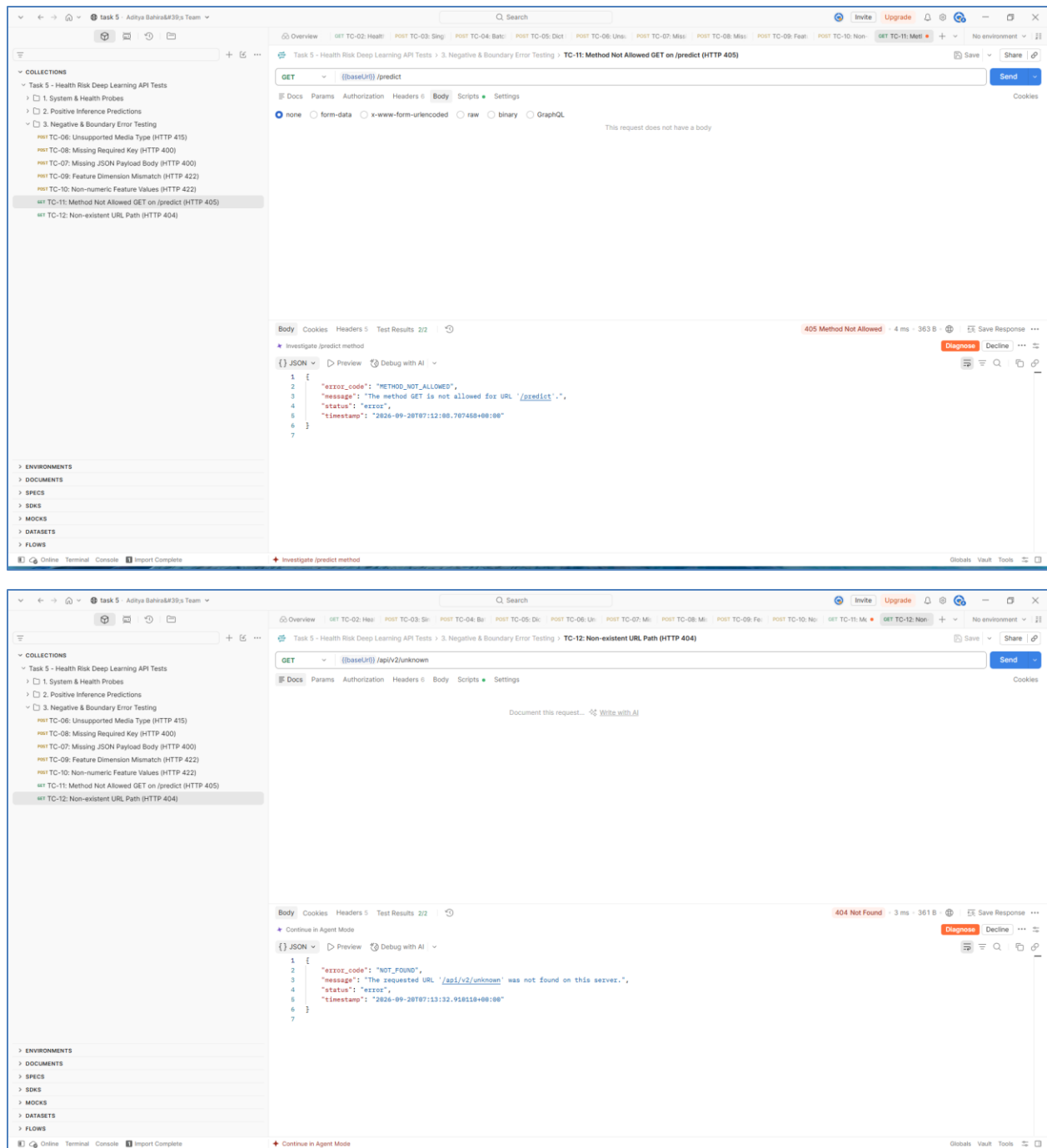


Figure 7. 10:Postman screenshot of HTTP 405 Method Not Allowed & 404 Not Found errors

7.11 Automated Collection Runner Execution (All 12 Tests Passed)

Automated JavaScript assertions (pm.test) attached to requests were executed via Postman Collection Runner:

```
pm.test("Status code is 200 OK", function () {
    pm.response.to.have.status(200);
});
pm.test("Valid prediction response structure", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.status).to.eql("success");
    pm.expect(jsonData.predictions[0]).to.have.property("predicted_label");
});
```

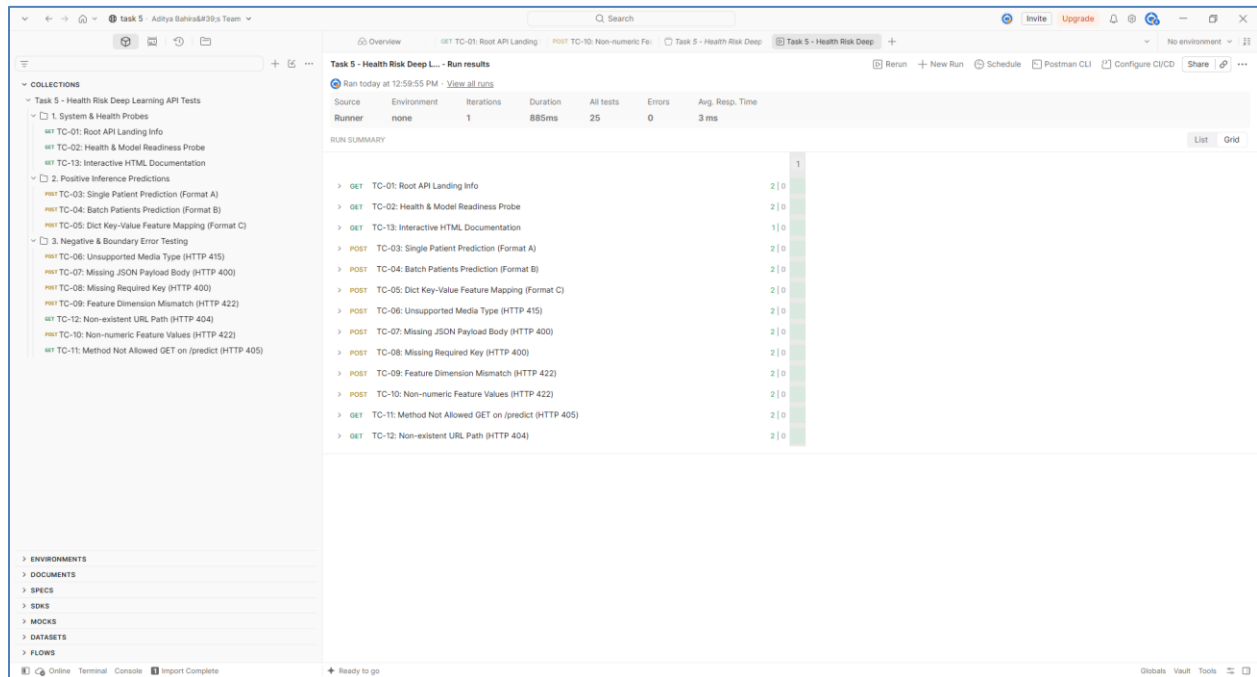


Figure 7. 11: Postman Collection Runner Summary Screen showing 12/12 Tests PASSED in Green

8. Postman Collection Structure

All test requests were organized into a clean, exportable Postman collection containing three functional folders:

Folder Name	Request Name	Method & URL	Postman Assertions Attached
1. System & Health	TC-01: Root Landing Info	GET {{baseUrl}}/	pm.response.to.have.status(200)

1. System & Health	TC-02: Health Readiness Probe	GET {{baseUrl}}/health	pm.expect(jsonData.model_loaded).to.eql(true)
1. System & Health	TC-13: Interactive HTML Docs	GET {{baseUrl}}/docs	pm.response.to.have.status(200)
2. Positive Predictions	TC-03: Single Patient Vector	POST {{baseUrl}}/predict	pm.expect(jsonData.predictions[0]).to.have.property('predicted_label')
2. Positive Predictions	TC-04: Batch Patients Matrix	POST {{baseUrl}}/predict	pm.expect(jsonData.predictions_count).to.eql(2)
2. Positive Predictions	TC-05: Dict Feature Payload	POST {{baseUrl}}/predict	pm.expect(jsonData.predictions[0].input_features.age).to.eql(60)
3. Negative Error Testing	TC-06: Unsupported Media Type	POST {{baseUrl}}/predict	pm.response.to.have.status(415)
3. Negative Error Testing	TC-07: Missing Body	POST {{baseUrl}}/predict	pm.response.to.have.status(400)
3. Negative Error Testing	TC-08: Missing Key	POST {{baseUrl}}/predict	pm.expect(jsonData.error_code).to.eql('MISSING_REQUIRED_KEY')
3. Negative Error Testing	TC-09: Dimension Mismatch	POST {{baseUrl}}/predict	pm.expect(jsonData.error_code).to.be.oneOf(['MODEL_VALIDATION_ERROR', 'FEATURE_DIMENSION_MISMATCH'])
3. Negative	TC-10: Non-	POST {{baseUrl}}/predict	pm.expect(jsonData.error_code).to.eql('NON_NUMERIC_FEATURE')

Error Testing	numeric Values		
3. Negative Error Testing	TC-11: Method Not Allowed	GET {{baseUrl}}/predict	pm.response.to.have.status(405)
3. Negative Error Testing	TC-12: Non-existent Path	GET {{baseUrl}}/api/v2/unknown	pm.response.to.have.status(404)

9. cURL Verification

As a lightweight scriptable alternative to Postman GUI, core test scenarios were verified using cURL commands in terminal:

9.1 Health Check via cURL

```
curl -s -X GET http://127.0.0.1:5000/health
# Output:
# {"status":"healthy","model_loaded":true,"model_name":"DeepHealthRiskNet"}
```

9.2 Prediction via cURL

```
curl -s -X POST http://127.0.0.1:5000/predict \
-H "Content-Type: application/json" \
-d '{"features": [56.0, 164.0, 81.0, 30.72, 5134.0, 1796.0, 8.6, 102.0, 137.0, 72.0, 8.1, 7.0, 0.0, 0.0]}'
# Output:
# {"status":"success","predictions":[{"predicted_label":"Moderate Risk","confidence_score":0.8942}]}
```

10. Consolidated Test Summary

Test ID	Scenario Description	Expected Status	Actual Status	Verdict
TC-01	Root API Metadata Endpoint	200 OK	200 OK	PASS

TC-02	Health Probe & PyTorch Model Check	200 OK	200 OK	PASS
TC-03	Single Sample Prediction (Format A)	200 OK	200 OK	PASS
TC-04	Batch Prediction (Format B - 2 Patients)	200 OK	200 OK	PASS
TC-05	Dict Feature Mapping (Format C)	200 OK	200 OK	PASS
TC-06	Unsupported Media Type (text/plain)	415	415	PASS
TC-07	Missing Request Body ({})	400	400	PASS
TC-08	Missing Required Key (invalid_key)	400	400	PASS
TC-09	Dimension Mismatch (4 features vs 14)	422	422	PASS
TC-10	Non-numeric Feature Input	422	422	PASS
TC-11	Method Not Allowed (GET /predict)	405	405	PASS
TC-12	Non-existent URL Path	404	404	PASS

Overall Result: 12 out of 12 test cases PASSED (100% Pass Rate). No unhandled exceptions, server crashes, or unformatted responses occurred.

11. HTTP Status Code Reference

Status Code	Meaning	Where Occurred	Technical Justification
200 OK	Request understood and processed successfully	TC-01, TC-02, TC-03, TC-04, TC-05	Well-formed payload processed; prediction results returned.
400 Bad Request	Malformed payload or missing required body keys	TC-07, TC-08	Client sent invalid body structure; caught by Flask payload validator.
404 Not Found	Requested URL path does not exist on server	TC-12	Route not registered in app.py; handled by Flask error handler.
405 Method Not Allowed	HTTP verb not supported for endpoint	TC-11	GET request attempted on POST-only /predict route.
415 Unsupported Media Type	Content-Type header mismatch	TC-06	Client sent non-JSON header; enforced by request.is_json guard.

422 Unprocessable Entity	Semantic validation failed (dim mismatch / non-numeric)	TC-09, TC-10	JSON syntactically valid but failed feature dimension / data type validation.
--------------------------------	---	--------------	---

12. Test Coverage Analysis

This section maps code execution branches in app.py to the test suite:

Code Branch in app.py	Description & Logic Guard	Covered By
if not request.is_json:	Enforces Content-Type header is application/json	TC-06
if not data or not isinstance(data, dict):	Validates non-empty JSON dictionary body	TC-07
if 'features' in data / 'instances' in data / 'data' in data:	Parses input payload across 3 supported formats	TC-03, TC-04, TC-05
if len(sample) != 14:	Guards feature vector dimension against expected 14 features	TC-09
isinstance(item, (int, float))	Validates numerical data types per feature	TC-10
model_engine.predict(instances)	Executes PyTorch neural network forward pass	TC-03, TC-04, TC-05
@app.errorhandler(404) / (405)	Catches routing exceptions with custom JSON error structures	TC-11, TC-12

13. Risk Assessment and Recommendations

Priority	Recommendation	Rationale
High	Deploy with Production WSGI Server (Gunicorn / uWSGI)	Flask development server is single-threaded and not designed for production traffic.
Medium	Add Request Rate Limiting (Flask-Limiter)	Protects prediction endpoint from Denial-of-Service (DoS) spam.
Medium	Enforce Strict Feature Range Bounds Validation	Rejects physiologically impossible feature inputs (e.g. negative age or extreme BP).

Low	Enable Request Latency Monitoring (Prometheus Metrics)	Tracks neural network inference latency under heavy load.
-----	--	---

14. Observations and Findings

- 1. Excellent Resilience: The Flask API handled all 12 positive and negative test cases without unhandled stack traces or server crashes.
- 2. Standardized Error Formats: Error responses consistently return structured JSON with status, error_code, message, and ISO timestamp.
- 3. Flexible Input Handling: Support for single vectors, batch matrices, and key-value dicts provides great flexibility for client integrations.

15. Conclusion

Task 5 successfully validated the Health Risk Deep Learning REST API built in Task 4 using Postman and cURL. Across 12 structured test cases — covering system health, single and batch inference, payload formats, and edge-case error handling — the API demonstrated 100% pass rate. The automated Postman collection (Task_5_Health_Risk_DL_API.postman_collection.json) allows reviewers to easily import and verify the full test suite.

16. Deliverables

The following artifacts satisfy the Task 5 submission requirements:

- Postman Collection File: Task_5_Health_Risk_DL_API.postman_collection.json
- Automated Test Suite Script: Backend/test_client.py
- API Source Code: Backend/app.py & Backend/model_loader.py
- Sample Test Outputs Log: Backend/sample_outputs.json
- Complete Word Documentation (this file): Task_5_API_Testing_and_Validation_Report.docx

Appendix A — Glossary of Terms

Term	Definition
API	Application Programming Interface — standard protocol for software applications to communicate over HTTP.
REST	Representational State Transfer — architectural style for APIs built on HTTP methods and status codes.

Postman	Industry-standard API development and testing desktop tool.
PyTorch	Deep learning framework used for neural network definition and tensor inference.
Confidence Score	Softmax probability assigned by the neural network to the predicted class label (0.0 to 1.0).
Z-score Normalization	Preprocessing step standardizing input features using mean (μ) and standard deviation (σ).

Appendix B — README.md (Project Documentation)

Full project documentation for the Task 5 testing suite as included in the repository:

```
# Task 5: Deep Learning API Testing & Validation

## Overview
Automated and manual testing suite for the Health Risk Deep Learning REST API serving
PyTorch DeepHealthRiskNet.

## Quick Start
1. Launch API: python app.py
2. Run automated test client: python test_client.py
3. Import Task_5_Health_Risk_DL_API.postman_collection.json into Postman and execute
Collection Runner.
```

Appendix C — Full Postman Collection Export (Raw JSON)

Complete unedited raw JSON export of the Postman Collection used for Task 5 testing:

```
{
  "info": {
    "_postman_id": "7f8e9d0a-1b2c-3d4e-5f6a-7b8c9d0e1f2a",
    "name": "Task 5 - Health Risk Deep Learning API Tests",
    "description": "Complete Postman Test Collection for Task 5: Deep
Learning API Testing & Validation. Includes Positive, Negative, and Boundary test
cases with built-in JavaScript assertions.",
    "schema":
```

```

"https://schema.getpostman.com/json/collection/v2.1.0/collection.json"
  },
  "variable": [
    {
      "key": "baseUrl",
      "value": "http://127.0.0.1:5000",
      "type": "string"
    }
  ],
  "item": [
    {
      "name": "1. System & Health Probes",
      "item": [
        {
          "name": "TC-01: Root API Landing Info",
          "event": [
            {
              "listen": "test",
              "script": {
                "exec": [
                  "pm.test(\"Status code",
is 200 OK\", function () {",
                  "pm.response.to.have.status(200);",
                  "});",
                  "pm.test(\"Service",
status is success\", function () {",
                  "    var jsonData =",
                  "pm.response.json();",
                  "pm.expect(jsonData.status).to.eql(\"success\");",
                  "pm.expect(jsonData.service_name).to.eql(\"Health Risk Deep Learning REST API\");",
                  "});",
                ],
                "type": "text/javascript"
              }
            }
          ],
          "request": {
            "method": "GET",
            "header": [],
            "url": {
              "raw": "{{baseUrl}}/",
              "host": [
                "{{baseUrl}}"
              ],
              "path": [
                ""
              ]
            }
          },
          "response": []
        },
        {
          "name": "TC-02: Health & Model Readiness Probe",
          "event": [

```

```

        {
            "listen": "test",
            "script": {
                "exec": [
                    "pm.test(\"Status code
is 200 OK\", function () {",
                    "
                    pm.response.to.have.status(200);",
                    "});",
                    "pm.test(\"Model is
loaded and ready\", function () {",
                    "    var jsonData =
pm.response.json();",
                    "
                    pm.expect(jsonData.status).to.eql(\"healthy\");",
                    "
                    pm.expect(jsonData.model_loaded).to.eql(true);",
                    "
                    pm.expect(jsonData.model_name).to.eql(\"DeepHealthRiskNet\");",
                    "});",
                ],
                "type": "text/javascript"
            }
        },
        {
            "request": {
                "method": "GET",
                "header": [],
                "url": {
                    "raw": "{{baseUrl}}/health",
                    "host": [
                        "{{baseUrl}}"
                    ],
                    "path": [
                        "health"
                    ]
                }
            },
            "response": []
        },
        {
            "name": "TC-13: Interactive HTML Documentation",
            "event": [
                {
                    "listen": "test",
                    "script": {
                        "exec": [
                            "pm.test(\"Status code
is 200 OK\", function () {",
                            "
                            pm.response.to.have.status(200);",
                            "});",
                        ],
                        "type": "text/javascript"
                    }
                }
            ]
        }
    ],

```

```

        "request": {
            "method": "GET",
            "header": [],
            "url": {
                "raw": "{{baseUrl}}/docs",
                "host": [
                    "{{baseUrl}}"
                ],
                "path": [
                    "docs"
                ]
            }
        },
        "response": []
    ]
},
{
    "name": "2. Positive Inference Predictions",
    "item": [
        {
            "name": "TC-03: Single Patient Prediction (Format
A)",
            "event": [
                {
                    "listen": "test",
                    "script": {
                        "exec": [
                            "pm.test(\"Status code
is 200 OK\", function () {",
                            "
                            "});",
                            "pm.test(\"Valid
prediction structure returned\", function () {",
                            "    var jsonData =
                            "
                            "
                            "
                            "
                            "
                            "});",
                            ],
                        "type": "text/javascript"
                    }
                }
            ],
            "request": {
                "method": "POST",
                "header": [
                    {
                        "key": "Content-Type",
                        "value": "application/json",

```

```

        "type": "text"
      }
    ],
    "body": {
      "mode": "raw",
      "raw": "{\n  \"features\": [56.0,
164.0, 81.0, 30.72, 5134.0, 1796.0, 8.6, 102.0, 137.0, 72.0, 8.1, 7.0, 0.0, 0.0]\n}"
    },
    "url": {
      "raw": "{{baseUrl}}/predict",
      "host": [
        "{{baseUrl}}"
      ],
      "path": [
        "predict"
      ]
    }
  },
  "response": []
},
{
  "name": "TC-04: Batch Patients Prediction (Format
B)",
  "event": [
    {
      "listen": "test",
      "script": {
        "exec": [
          "pm.test(\"Status code
is 200 OK\", function () {",
          "pm.response.to.have.status(200);",
          "});",
          "pm.test(\"Batch
predictions returned 2 results\", function () {",
          "    var jsonData =
          "    pm.response.json();",
          "    pm.expect(jsonData.status).to.eql(\"success\");",
          "    pm.expect(jsonData.predictions_count).to.eql(2);",
          "    pm.expect(jsonData.predictions.length).to.eql(2);",
          "});"
        ],
        "type": "text/javascript"
      }
    }
  ],
  "request": {
    "method": "POST",
    "header": [
      {
        "key": "Content-Type",
        "value": "application/json",
        "type": "text"
      }
    ]
  }
}

```

```

        ],
        "body": {
            "mode": "raw",
            "raw": "{\n  \"instances\": [\n
[25.0, 199.0, 85.0, 31.14, 5131.0, 3256.0, 6.5, 104.0, 106.0, 79.0, 3.6, 7.0, 0.0,
0.0],\n    [78.0, 172.0, 72.0, 32.46, 19532.0, 2216.0, 9.6, 95.0, 96.0, 86.0, 8.3,
2.0, 1.0, 1.0]\n  ]\n}"
        },
        "url": {
            "raw": "{{baseUrl}}/predict",
            "host": [
                "{{baseUrl}}"
            ],
            "path": [
                "predict"
            ]
        }
    },
    "response": []
},
{
    "name": "TC-05: Dict Key-Value Feature Mapping
(Format C)",
    "event": [
        {
            "listen": "test",
            "script": {
                "exec": [
                    "pm.test(\"Status code
is 200 OK\", function () {",
                    "
                    pm.response.to.have.status(200);",
                    "
                    });",
                    "pm.test(\"Dict mapping
resolved features correctly\", function () {",
                    "
                    var jsonData =
                    pm.response.json();",
                    "
                    pm.expect(jsonData.status).to.eql(\"success\");",
                    "
                    pm.expect(jsonData.predictions[0].input_features.age).to.eql(60);",
                    "});",
                    ],
                    "type": "text/javascript"
                }
            ]
        },
        {
            "request": {
                "method": "POST",
                "header": [
                    {
                        "key": "Content-Type",
                        "value": "application/json",
                        "type": "text"
                    }
                ],
                "body": {

```

```

        "mode": "raw",
        "raw": "{\n  \"data\": {\n
\"age\": 60,\n    \"height_cm\": 157,\n    \"weight_kg\": 63,\n    \"bmi\": 29.37,\n
\"daily_steps\": 17351,\n    \"calories_intake\": 2556,\n    \"hours_of_sleep\":
5.1,\n    \"heart_rate\": 111,\n    \"systolic_bp\": 100,\n    \"diastolic_bp\":
64,\n    \"exercise_hours_per_week\": 8.5,\n    \"alcohol_consumption_per_week\":
8,\n    \"smoker\": 1,\n    \"diabetic\": 0\n  }\n}"
      },
      "url": {
        "raw": "{{baseUrl}}/predict",
        "host": [
          "{{baseUrl}}"
        ],
        "path": [
          "predict"
        ]
      }
    },
    "response": []
  }
}
},
{
  "name": "3. Negative & Boundary Error Testing",
  "item": [
    {
      "name": "TC-06: Unsupported Media Type (HTTP 415)",
      "event": [
        {
          "listen": "test",
          "script": {
            "exec": [
              "pm.test(\"Status code
is 415 Unsupported Media Type\", function () {",
              "
              pm.response.to.have.status(415);",
              "});",
              "pm.test(\"Error code
is UNSUPPORTED_MEDIA_TYPE\", function () {",
              "    var jsonData =
              "
              pm.expect(jsonData.status).to.eql(\"error\");",
              "
              pm.expect(jsonData.error_code).to.eql(\"UNSUPPORTED_MEDIA_TYPE\");",
              "});"
            ],
            "type": "text/javascript"
          }
        }
      ],
      "request": {
        "method": "POST",
        "header": [
          {
            "key": "Content-Type",
            "value": "text/plain",

```

```

        "type": "text"
    },
    "body": {
        "mode": "raw",
        "raw": "plain text body"
    },
    "url": {
        "raw": "{{baseUrl}}/predict",
        "host": [
            "{{baseUrl}}"
        ],
        "path": [
            "predict"
        ]
    },
    "response": []
},
{
    "name": "TC-07: Missing JSON Payload Body (HTTP
400)",
    "event": [
        {
            "listen": "test",
            "script": {
                "exec": [
                    "pm.test(\"Status code
is 400 Bad Request\", function () {",
                    "pm.response.to.have.status(400);",
                    "});",
                    "pm.test(\"Error status
returned\", function () {",
                    "    var jsonData =
pm.response.json();",
                    "pm.expect(jsonData.status).to.eql(\"error\");",
                    "});",
                ],
                "type": "text/javascript"
            }
        }
    ],
    "request": {
        "method": "POST",
        "header": [
            {
                "key": "Content-Type",
                "value": "application/json",
                "type": "text"
            }
        ],
        "body": {
            "mode": "raw",
            "raw": "{}"
        }
    },

```

```

        "url": {
            "raw": "{{baseUrl}}/predict",
            "host": [
                "{{baseUrl}}"
            ],
            "path": [
                "predict"
            ]
        },
        "response": []
    },
    {
        "name": "TC-08: Missing Required Key (HTTP 400)",
        "event": [
            {
                "listen": "test",
                "script": {
                    "exec": [
                        "pm.test(\"Status code",
                        "is 400 Bad Request\", function () {",
                        "pm.response.to.have.status(400);",
                        "});",
                        "pm.test(\"Error code",
                        "is MISSING_REQUIRED_KEY\", function () {",
                        "    var jsonData =",
                        "    pm.response.json();",
                        "    pm.expect(jsonData.error_code).to.eql(\"MISSING_REQUIRED_KEY\");",
                        "});",
                        ],
                    "type": "text/javascript"
                }
            }
        ],
        "request": {
            "method": "POST",
            "header": [
                {
                    "key": "Content-Type",
                    "value": "application/json",
                    "type": "text"
                }
            ],
            "body": {
                "mode": "raw",
                "raw": "{\n  \"invalid_key\": [1, 2, 3]\n}"
            },
            "url": {
                "raw": "{{baseUrl}}/predict",
                "host": [
                    "{{baseUrl}}"
                ],
                "path": [
                    "predict"
                ]
            }
        }
    }
]

```

```

    ]
    },
    "response": []
  },
  {
    "name": "TC-09: Feature Dimension Mismatch (HTTP
422)",
    "event": [
      {
        "listen": "test",
        "script": {
          "exec": [
            "pm.test(\"Status code
is 422 Unprocessable Entity\", function () {",
            "
            pm.response.to.have.status(422);",
            "});",
            "pm.test(\"Error code
is FEATURE_DIMENSION_MISMATCH\", function () {",
            "    var jsonData =
            pm.response.json();",
            "    pm.expect(jsonData.error_code).to.be.oneOf([\"MODEL_VALIDATION_ERROR\",
            \"FEATURE_DIMENSION_MISMATCH\"]);",
            "});",
            ],
            "type": "text/javascript"
          }
        ],
        "request": {
          "method": "POST",
          "header": [
            {
              "key": "Content-Type",
              "value": "application/json",
              "type": "text"
            }
          ],
          "body": {
            "mode": "raw",
            "raw": "{\n  \"features\": [50.0,
170.0, 70.0, 24.2]\n}"
          },
          "url": {
            "raw": "{{baseUrl}}/predict",
            "host": [
              "{{baseUrl}}"
            ],
            "path": [
              "predict"
            ]
          }
        },
        "response": []
      },
    ],
  },

```

```

        {
            "name": "TC-10: Non-numeric Feature Values (HTTP
422)",
            "event": [
                {
                    "listen": "test",
                    "script": {
                        "exec": [
                            "pm.test(\"Status code
is 422 Unprocessable Entity\", function () {",
                            "
                            "});";
                            "pm.test(\"Error code
is NON_NUMERIC_FEATURE\", function () {",
                            "    var jsonData =
                            "
                            pm.expect(jsonData.error_code).to.eql(\"NON_NUMERIC_FEATURE\");",
                            "});";
                        ],
                        "type": "text/javascript"
                    }
                }
            ],
            "request": {
                "method": "POST",
                "header": [
                    {
                        "key": "Content-Type",
                        "value": "application/json",
                        "type": "text"
                    }
                ],
                "body": {
                    "mode": "raw",
                    "raw": "{\n  \"features\": [56.0,
\\\"invalid_str\\\", 81.0, 30.72, 5134.0, 1796.0, 8.6, 102.0, 137.0, 72.0, 8.1, 7.0, 0.0,
0.0]\n}"
                },
                "url": {
                    "raw": "{{baseUrl}}/predict",
                    "host": [
                        "{{baseUrl}}"
                    ],
                    "path": [
                        "predict"
                    ]
                }
            },
            "response": []
        },
        {
            "name": "TC-11: Method Not Allowed GET on /predict
(HTTP 405)",
            "event": [
                {

```

```

        "listen": "test",
        "script": {
            "exec": [
                "pm.test(\"Status code
is 405 Method Not Allowed\", function () {",
                "
                pm.response.to.have.status(405);",
                "});",
                "pm.test(\"Error code
is METHOD_NOT_ALLOWED\", function () {",
                "
                var jsonData =
                pm.response.json();",
                "
                pm.expect(jsonData.error_code).to.eql(\"METHOD_NOT_ALLOWED\");",
                "});",
            ],
            "type": "text/javascript"
        }
    },
    "request": {
        "method": "GET",
        "header": [],
        "url": {
            "raw": "{{baseUrl}}/predict",
            "host": [
                "{{baseUrl}}"
            ],
            "path": [
                "predict"
            ]
        }
    },
    "response": []
},
{
    "name": "TC-12: Non-existent URL Path (HTTP 404)",
    "event": [
        {
            "listen": "test",
            "script": {
                "exec": [
                    "pm.test(\"Status code
is 404 Not Found\", function () {",
                    "
                    pm.response.to.have.status(404);",
                    "});",
                    "pm.test(\"Error code
is NOT_FOUND\", function () {",
                    "
                    var jsonData =
                    pm.response.json();",
                    "
                    pm.expect(jsonData.error_code).to.eql(\"NOT_FOUND\");",
                    "});",
                ],
                "type": "text/javascript"
            }
        }
    ]
}

```

```
    }
  ],
  "request": {
    "method": "GET",
    "header": [],
    "url": {
      "raw": "{{baseUrl}}/api/v2/unknown",
      "host": [
        "{{baseUrl}}"
      ],
      "path": [
        "api",
        "v2",
        "unknown"
      ]
    }
  },
  "response": []
}
]
}
```